

AI Agents in VS Code for Academic Work

- Nim Dvir, PhD, MBA
- University at Albany
- Cengage Computing Experience 2026



2026

Cengage Computing Experience

Future-Focused Education,
Career-Ready Graduates



AI Agents in VS Code for Academic Work



From Prompting to Workflows · Nim Dvir, PhD,
MBA · University at Albany, Massry School
of Business



Scan to follow along

About me

- I teach databases, SQL, and analytics.
- I build courseware, websites, and a textbook.
- ndvir@albany.edu
- linkedin.com/in/nimdvir
- albany.edu/business/faculty/nim-dvir



What are we going to do today

- Learn about agents and how to use them
- Go over how to set up the workspace
- Learn about the workflow, including using prompts, skills and custom instruction
- Go over examples for educator

Why are we doing this?

I always wanted to build things but didn't feel I had the knowledge — without being an engineer, I thought I couldn't do it. Then AI agents changed everything. Now I build websites, scripts, automations — you name it — and I'm happy to share that knowledge.

What can agents do?

- [The website](#)
- The syllabus
- Data analysis
- The courseware



What are agents?

- AI that works toward a goal in steps, using tools, instead of only answering.
- Work locally

An agent follows a simple loop:

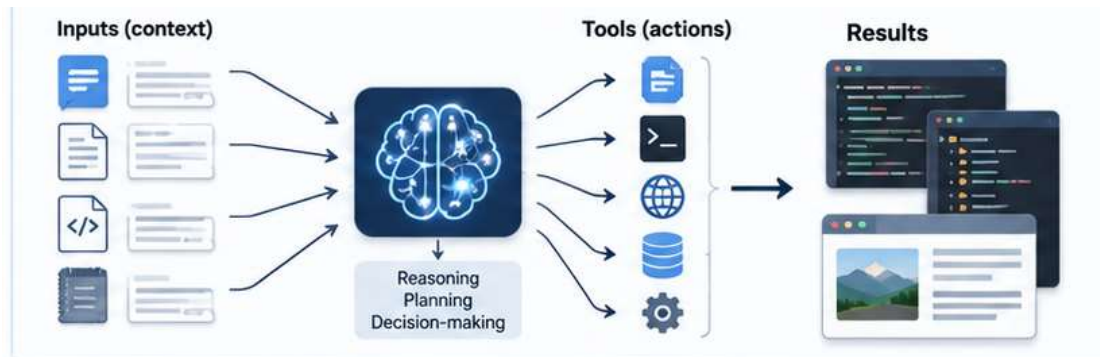
Goal → Inspect → Plan → Act → Check → Revise



| Chatbot

- A conversational interface that responds to messages.
- Chat is conversational. It is excellent for questions, writing, and brainstorming. You carry the output back into your work, and you coordinate the steps yourself.
- Example: explain how to revise a syllabus.

An agent doesn't answer. It works.



An AI agent uses a model, context, instructions, and tools to work toward a goal through multiple steps.

Chat: you ask → it answers → you do the work

Agent: you give a goal → it inspects → plans → acts → checks

Chat is the interface. An agent is a way the AI can operate.

Chat vs. agent

Agent — AI that works toward a goal in steps, using tools, instead of only answering.

Chat — the conversation window you type into; it is the interface, not the agent.

CHAT

AGENT

You ask → AI answers → You do the work You give a goal → AI examines → plans → acts → checks

Agent starts from a goal. The agent may inspect files, search, create and edit files, run commands, analyze data, use tools, inspect results, and correct itself.

Chat is the interface. An agent is a way the AI can operate.

| Why I use agents

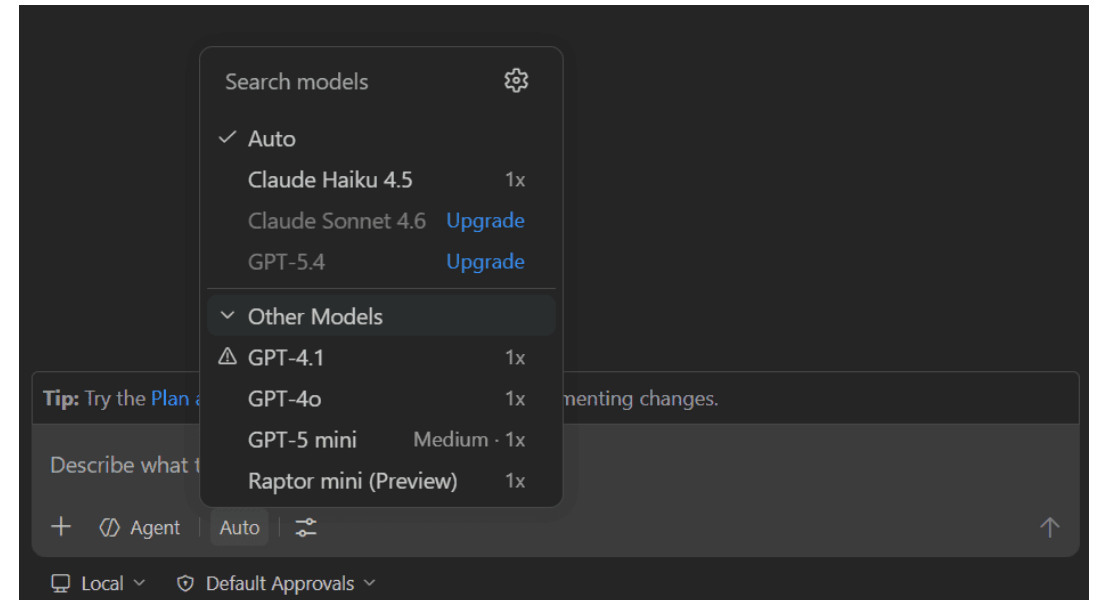
- They help me turn ideas into working materials.
- I can repeat academic tasks and inspect what changed.
- I still decide what is correct.

| Model

- The trained system that generates responses and proposes actions.
- An agent combines a model with context and tools.

Model choices

- Examples: GPT, Claude, and Gemini.
- Choose for the task, then check the result.
- [Model documentation](#)



| Tokens

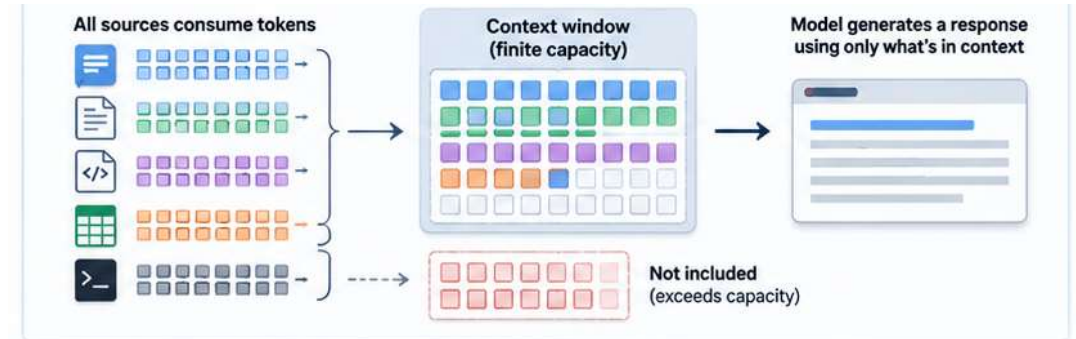
- Units a model processes in its input and output.
- A token may be a word, part of a word, or punctuation.

| Context

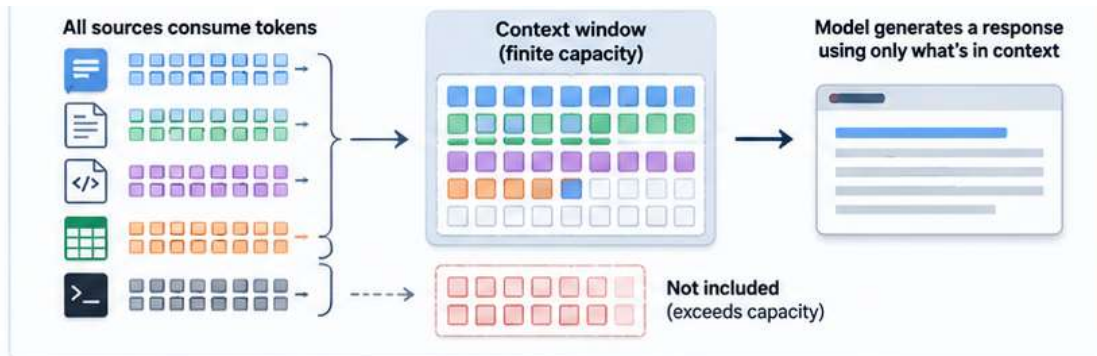
- The information available to the model for the current request.
- It can include instructions, chat, selected files, and tool results.
- [How context works](#)

Context window

- The maximum amount of context a model can process at once.
- It is limited. Relevant material must fit.



It can only see what you give it



Context, model, tokens

Context = prompt + conversation + files + instructions + tool results

Model = the engine

Tokens = the units it reads and writes; everything counts

Give it the material relevant to the task, not everything you have.

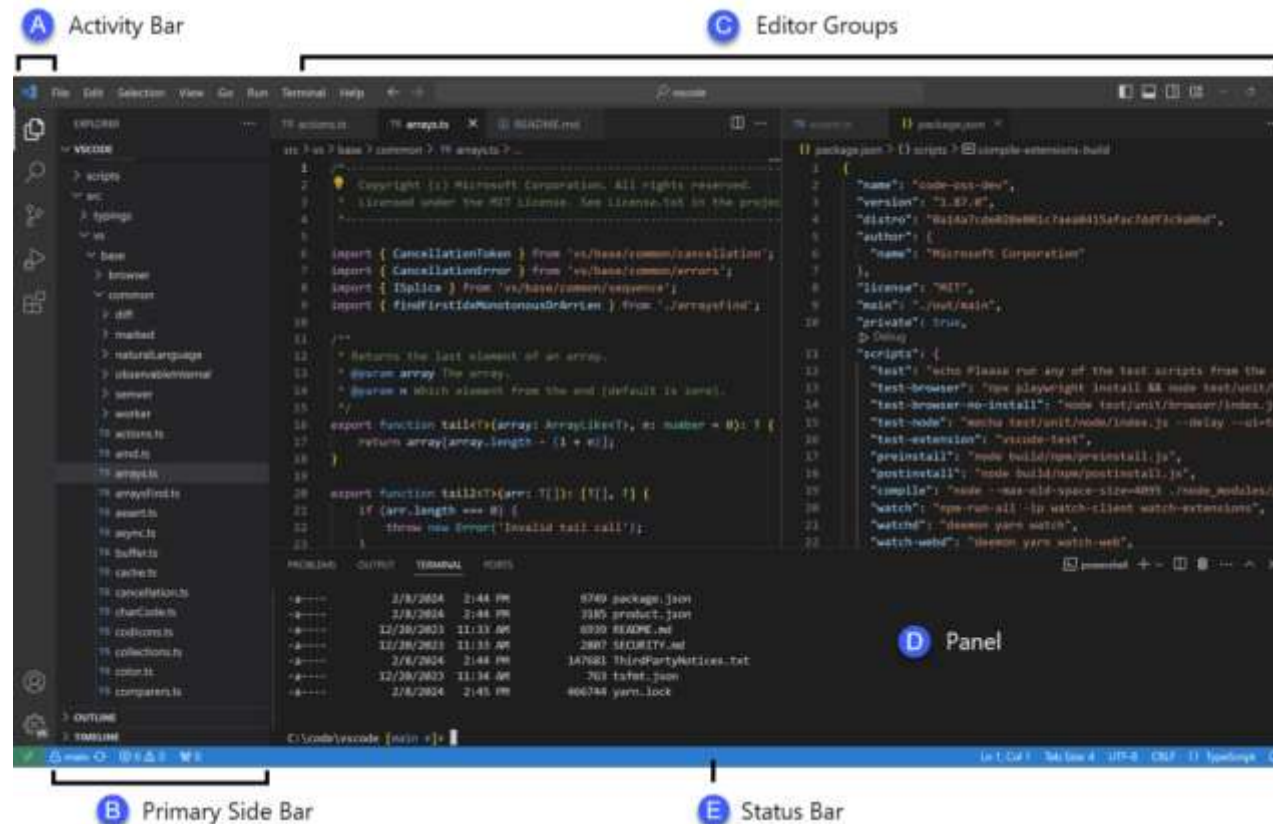
The workspace

| Workspace

- The project folders open in your editor.
- The workspace gives the agent a place to find inputs and save outputs.

VS Code: your files, with an AI beside them

VS Code + your files + Copilot = an agentic workspace • Setup takes about twenty minutes. Guide on the site.

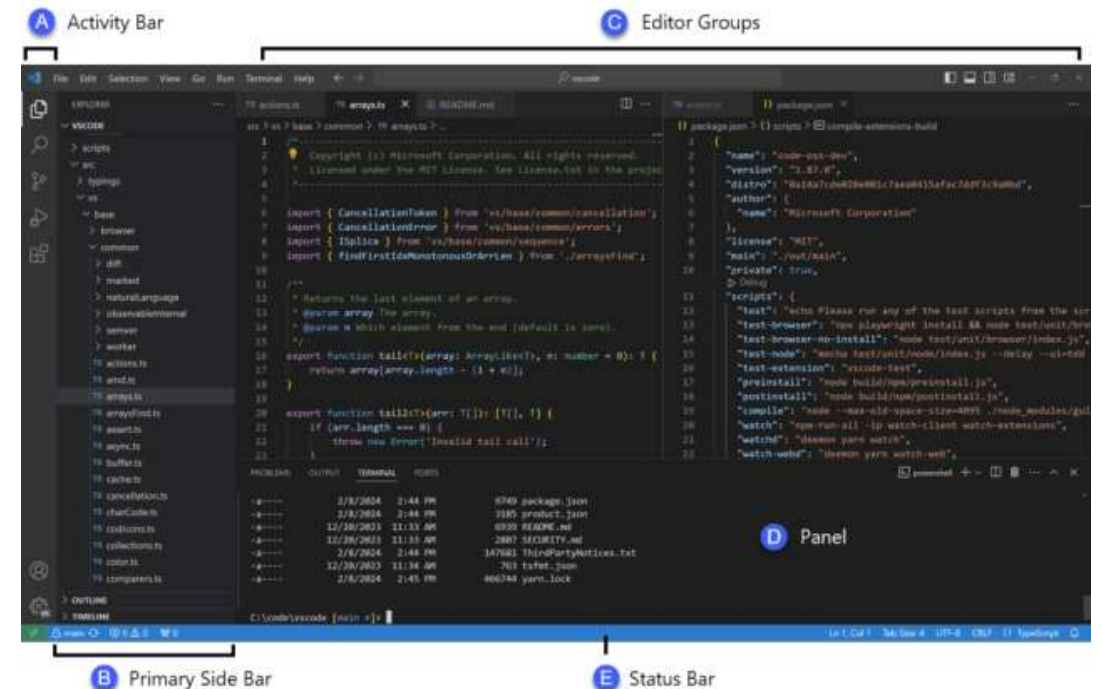


Text editor

- An application for reading and changing plain-text files.
- Markdown, HTML, and CSV can all be edited as text.

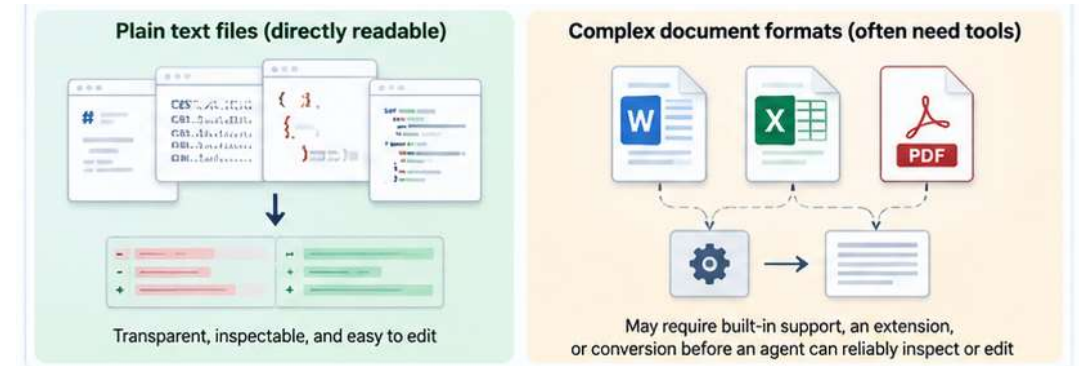
VS Code

- A text editor with project navigation, previews, and extensions.
- It is the workspace used in this session.
- [VS Code documentation](#)



File

- A named item that stores information.
- Examples: an outline.md, a syllabus.html, or attendance.csv.

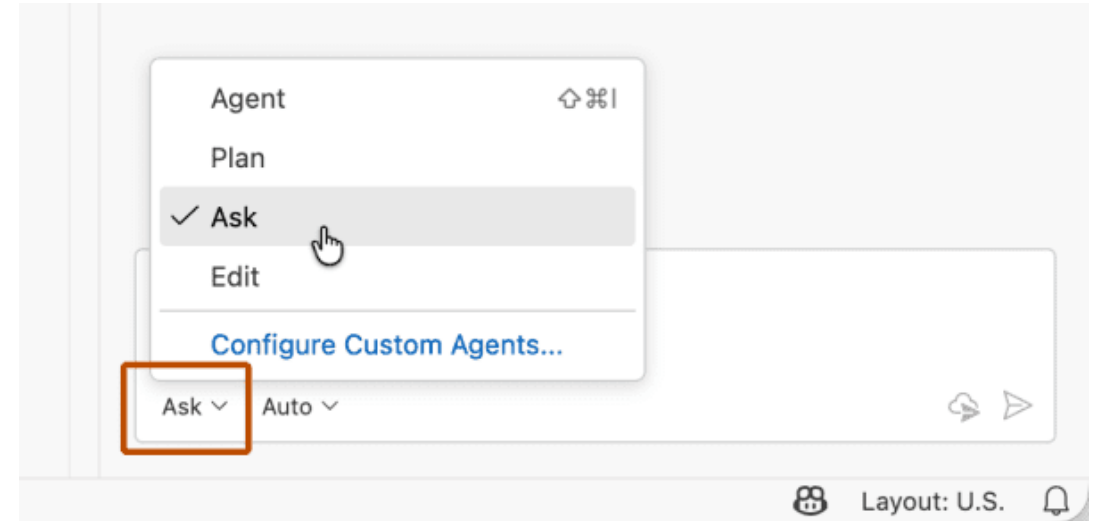


| Folder

- A container that organizes files and other folders.
- Keep sources, drafts, and finished outputs easy to distinguish.

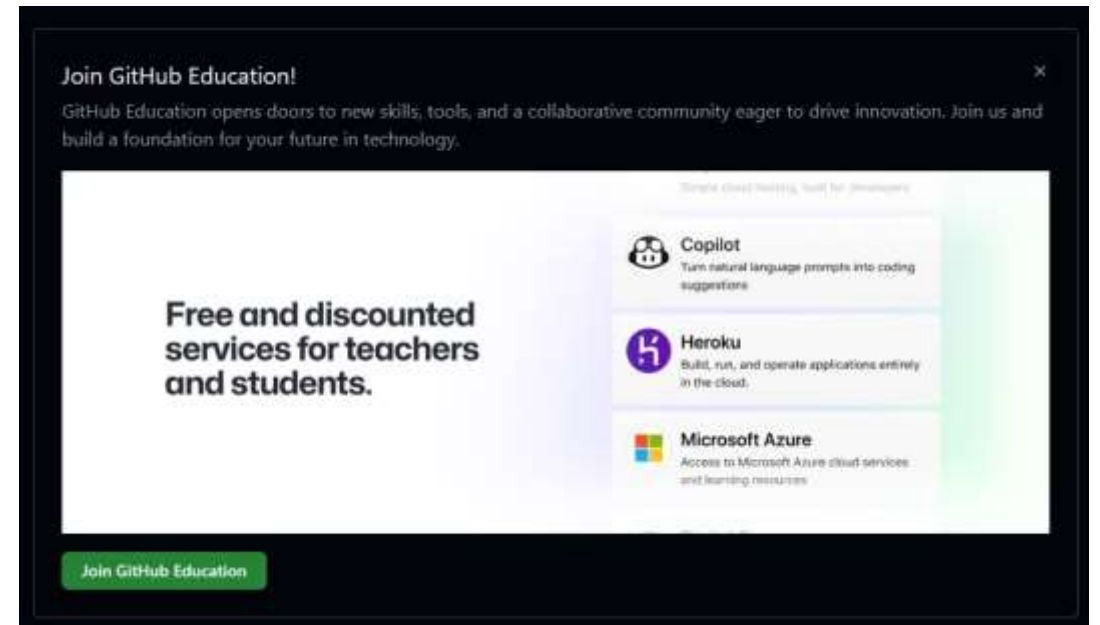
GitHub Copilot

- An AI assistant available inside VS Code.
- Ask, Plan, and Agent support different kinds of work.
- [GitHub Copilot documentation](#)



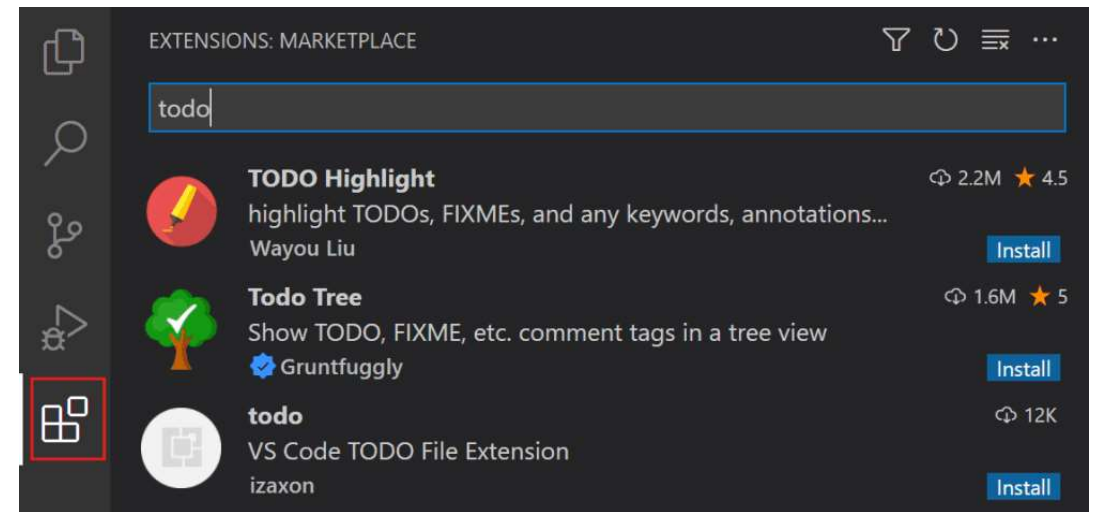
GitHub Education

- A program that provides benefits to eligible educators and students.
- Verified teachers can receive Copilot Pro at no cost.
- [Teacher eligibility and benefits](#)



Extension

- An add-on that gives VS Code another capability.
- Examples include AI assistance, previews, and language support.
- [Extension documentation](#)



Google Drive

- A service for storing and synchronizing files.
- My working files live in Drive.
- Make demo files available offline before presenting.
- [Open the CCE project folder](#)

| GitHub

- An online service that hosts Git repositories.
- I use it for versioning, sharing, and publishing websites.
- [CCE GitHub repository](#)

| Task

- The outcome you want the agent to produce.
- Define the source, constraints, output, and success check.

| Prompt

- The request and guidance you give the agent for a task.
- Save reusable task requests in a prompt folder.
- [Open the prompt folder](#)

Custom instructions

- Standing guidance that applies across requests.
- Project instructions describe this workspace.
- Personal instructions express your general preferences.
- [Custom instructions](#)

Skill

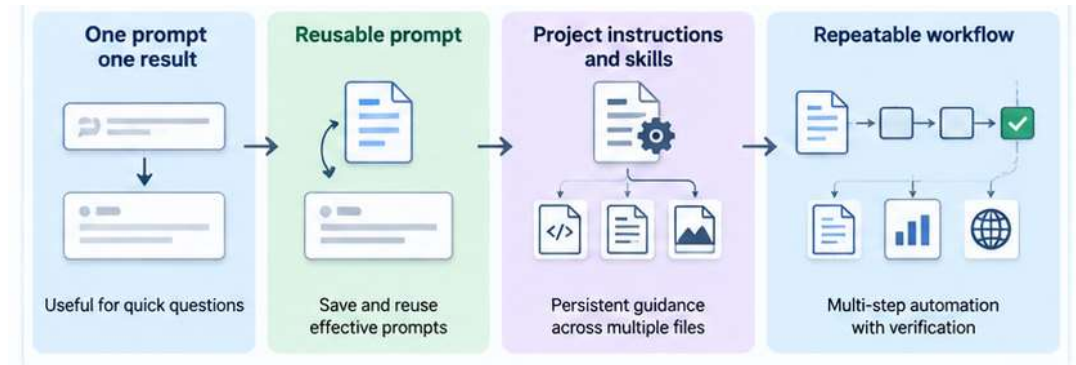
- A reusable procedure with instructions and optional supporting files.
- Example: check a syllabus for inconsistent dates and grading weights.
- [Skill documentation](#)

Project organization

- Sources and examples stay with the project.
- The prompts folder holds saved tasks.
- The skills folder holds reusable procedures.
- [Open prompts and skill examples](#)

Workflow

- A repeatable sequence for completing a task.
- Understand the source, plan, act, review, and verify.



Goal + Context + Constraints

Goal	Context	Constraints
✗ "Fix my course." ✓ "Update this syllabus and schedule for Spring 2027."	The files. Syllabus, schedule, policies, a CSV, the existing site.	Don't change grading weights. Don't invent dates. Flag uncertainty. Don't commit.

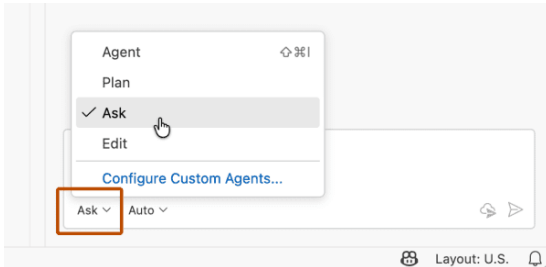
The course files are in live/syllabus/.
Update the course for Spring 2027 without changing grading weights. Identify inconsistent dates and flag anything requiring judgment.

Ask. Plan. Agent. Then watch the diff.

Ask

Understand.

No edits.



Plan

Propose the changes.

Waits for you.

Agent

Does the work.

Runs tools.

Review

You.

Files changed, the diff, Git status.

"Completed" does not necessarily mean "correct."

From prompting to workflows

The subtitle of this talk

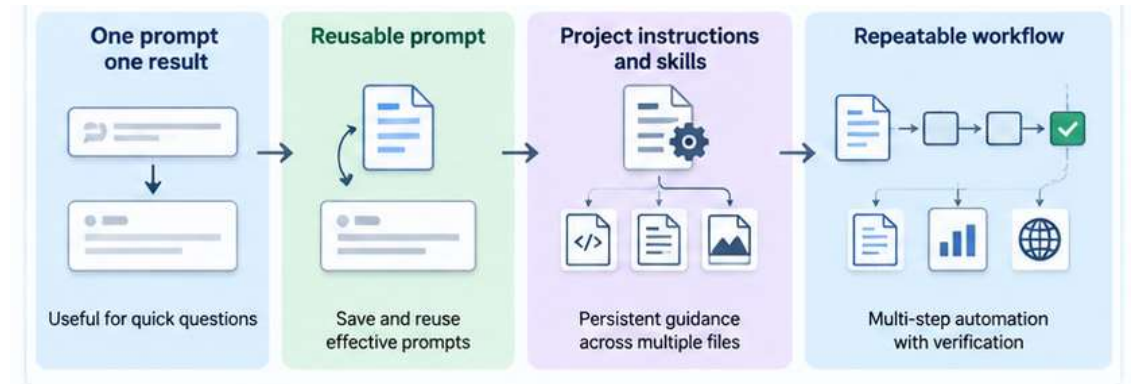
PROMPT — one task, typed once

REUSABLE PROMPT — `.github/prompts/update-syllabus.prompt.md`

PROJECT INSTRUCTIONS — `AGENTS.md`: "Always work this way in this project."

SKILL — `.github/skills/grade-submission/SKILL.md`

WORKFLOW — several steps, chained and repeatable

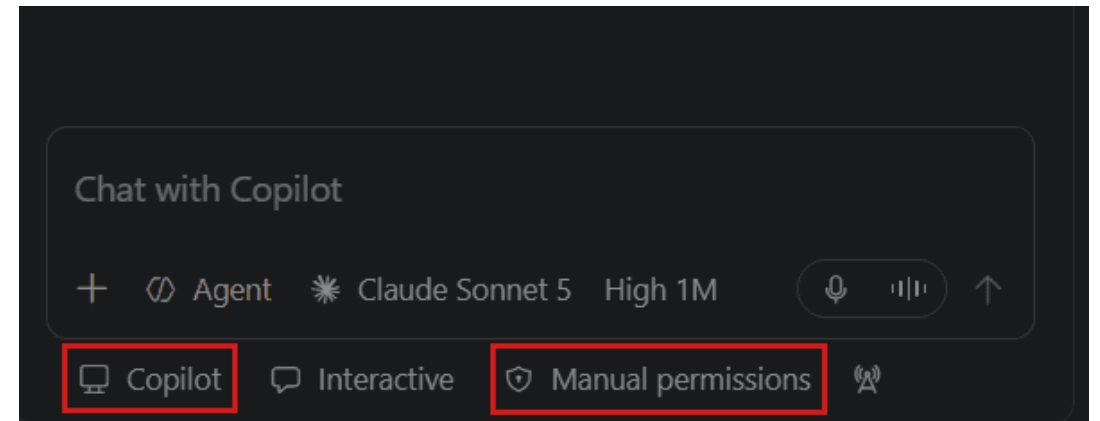


A complete request

- Goal: clarify one syllabus heading.
- Source: the current syllabus.
- Keep dates and grading unchanged.
- Save a copy and show the exact change.

My prepared setup

- Project open in VS Code
- Copilot signed in and model selected
- Outline, prompts, skills, and examples open



Examples

Recording to outline

- Start with the existing transcript.
- Organize the ideas in Markdown.
- Review the sequence before making slides.
- [Open the full lecture outline](#)

Outline to slides

- Use the reviewed outline and Cengage template.
- Keep one definition on each slide.
- Put your explanation and demo cues in speaker notes.

Slides and website

- The deck controls the lecture sequence.
- The website adds detail and take-home resources.
- Both follow the same reviewed outline.
- [Open the companion website](#)

How I built this website

The plumbing behind slide 2

outlines + prompts + template → agent → Markdown
→ build script → GitHub → GitHub Pages → your
phone

Clone · edit · commit · push.

Git keeps the history. GitHub keeps the copy. Pages
publishes the docs/ folder.

Nothing here needed a web developer.

Branch

Your GitHub Pages site is currently being built from the main branch. [Learn more.](#)

None ▼

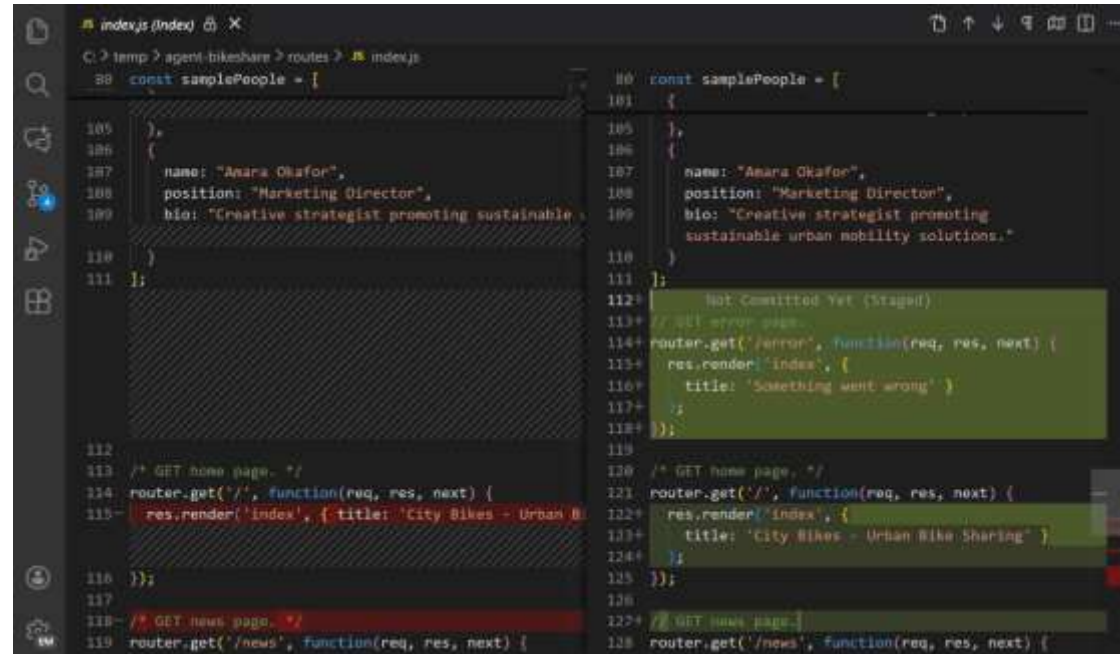
Save

Your syllabus, updated while you watch

4 course files with inconsistent dates → agent → updated files + a list of flagged decisions

/update-syllabus · Plan → Agent → diff

Watch for: reads all four files · proposes first · flags what needs judgment



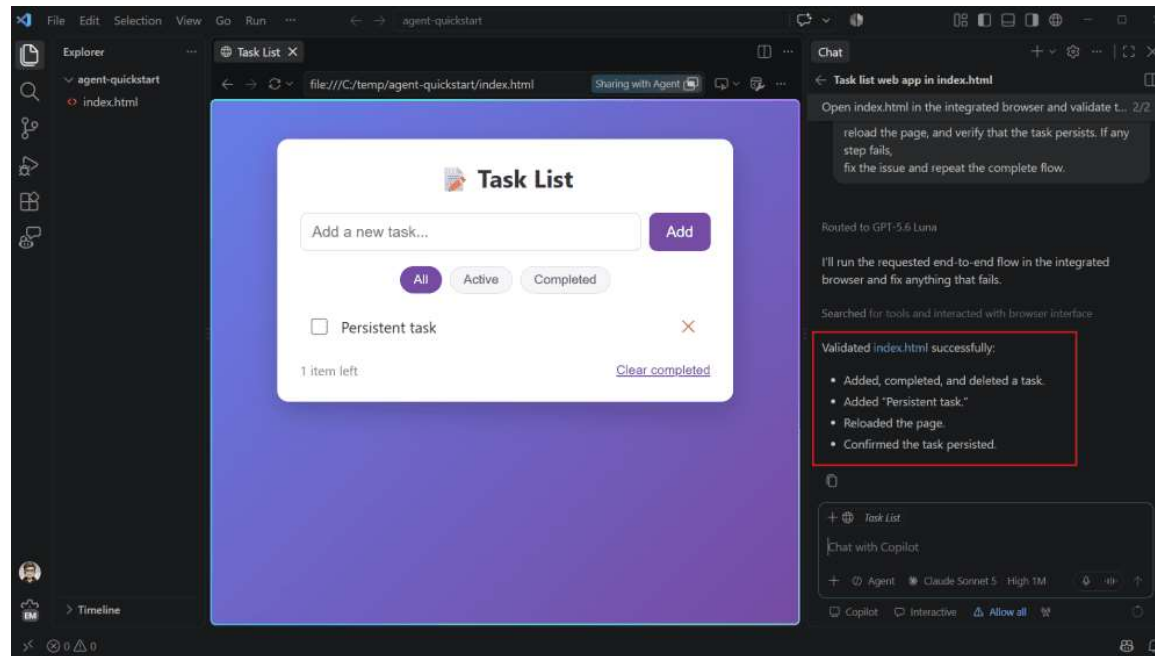
```
index.js (Index) X
C:\temp> agent-bikeshare > routes > index.js
88 const samplePeople = [
105   },
106   {
107     name: "Amara Okafor",
108     position: "Marketing Director",
109     bio: "Creative strategist promoting sustainable
110   }
111 ];
112
113 /* GET home page. */
114 router.get('/', function(req, res, next) {
115   res.render('index', {title: 'City Bikes - Urban B
116 });
117
118 /* GET news page. */
119 router.get('/news', function(req, res, next) {
120
121 const samplePeople = [
122   {
123     name: "Amara Okafor",
124     position: "Marketing Director",
125     bio: "Creative strategist promoting
    sustainable urban mobility solutions."
126   }
127 ];
128
129 // GET error page
130 router.get('/error', function(req, res, next) {
131   res.render('index', {
132     title: 'Something went wrong'
133   });
134 });
135
136 /* GET home page. */
137 router.get('/', function(req, res, next) {
138   res.render('index', {
139     title: 'City Bikes - Urban Bike Sharing'
140   });
141 });
142
143 // GET news page
144 router.get('/news', function(req, res, next) {
```

From a feedback spreadsheet to one chart and a paragraph

student-feedback.csv → inspect → analyze → visualize → interpret

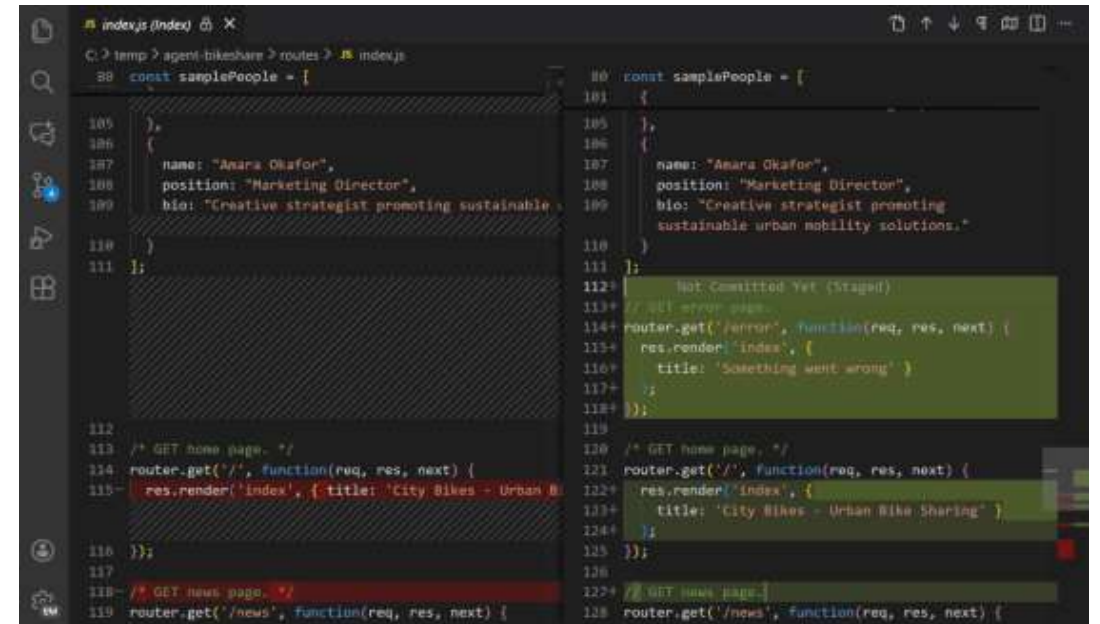
/analyze-feedback · Agent

Watch for: looks at the columns first · runs Python · one chart · "Do not invent data."



One small site edit

- Change a single banner sentence.
- Preview the local result.
- Inspect the diff before accepting it.



```
index.js (Index) X
C:\temp> agent-bikeshare > routes > .\index.js
88 const samplePeople = [
105   },
106   {
107     name: "Amara Okafor",
108     position: "Marketing Director",
109     bio: "Creative strategist promoting sustainable
110   }
111 ];
112
113 /* GET home page. */
114 router.get('/', function(req, res, next) {
115   res.render('index', { title: 'City Bikes - Urban B
116 });
117
118 /* GET news page. */
119 router.get('/news', function(req, res, next) {
120
121   // Not Committed Yet (staged)
122   // GET error page.
123   router.get('/error', function(req, res, next) {
124     res.render('index', {
125       title: 'Something went wrong'
126     });
127   });
128
129 /* GET home page. */
130 router.get('/', function(req, res, next) {
131   res.render('index', {
132     title: 'City Bikes - Urban Bike Sharing'
133   });
134 });
135
136 /* GET news page. */
137 router.get('/news', function(req, res, next) {
```

The syllabus source

- Use the current syllabus.html.
- Its internal update is September 28.
- Keep older versions as references.
- [Open the course syllabus](#)

The syllabus revision

- Before: Course Schedule
- After: Course Schedule and Deadlines
- Change only the heading.
- [Open the prepared examples](#)

| Syllabus checks

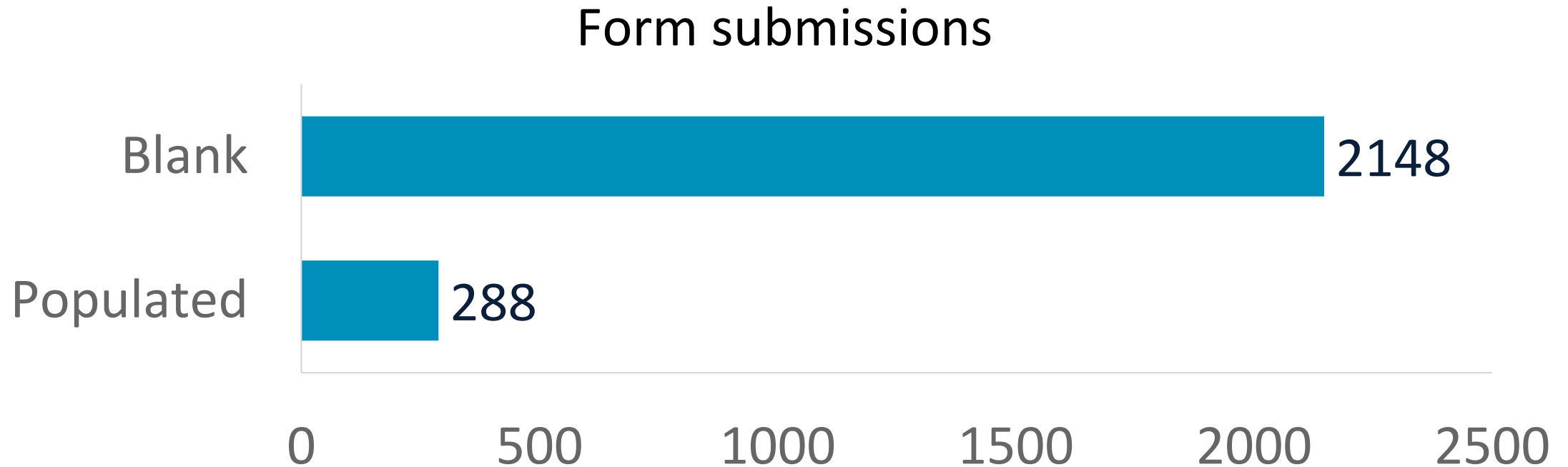
- Dates and policies stay unchanged.
- Each grading weight stays unchanged.
- The seven weights still total 100%.
- The change report shows the evidence.

The course data

- 2,436 form submissions and 9 columns.
- Check the unit of analysis and missing values.
- Reconcile dates before comparing class sessions.
- [Open the aggregate examples](#)

Participation field

- 2,436 submissions: 288 populated, 2,148 blank.



What the chart means

- The chart describes field completeness.
- A blank does not prove no participation.
- Submission counts do not identify unique students.

Reusable prompts

- Open a saved task prompt.
- Identify the source and expected output.
- Run it on a copy and check the result.
- [Open the eight prepared prompts](#)

Reusable skill example

- The syllabus consistency procedure checks each revision.
- Review its steps and inspect its findings.
- [Open the skill example folder](#)

SWOT

- Strength
- Weaknesses
- Opportunities
- Threats

Take it with you

On the site

These slides · setup guide · every prompt · the demo files · the repository · recordings

github.com/nimdvir/cce-2026

Nim Dvir · nimdvir.com · linkedin.com/in/nimdvir

Scan once more

QR code and short URL: TODO (add qr-site.png once the site URL is final)

Resources

- Lecture outline and speaker notes
- Prompts and skill examples



| Questions

- Which recurring task would you try?
- What would you check before using the result?

Thank you

- Nim Dvir
- University at Albany
- nimdvir.com

